



SQA FINAL TERM PROJECT

QA TECHNIQUES IN AI RESUME ANALYSIS

SUBMITTED BY:

<u>Name:</u>	<u>RollNo:</u>
M Huzaifa Asghar	Fa23/BSCS/617/A
Zunayra Junaid	Fa23/BSCS/632/A
Sadia Sultan	Fa23/BSCS/554/A
Eman Shaukat	Fa23/BSCS/006/A
Farhina Aslam	Fa23/BSCS/555/A

SUBMITTED TO:

MISS AMMARA KANWAL

6th Semester

DEFECT PREVENTION, DEFECT REDUCTION, AND DEFECT CONTAINMENT TECHNIQUES

1. Introduction

Software quality assurance aims to identify and eliminate defects throughout the software development life cycle. In the AI Resume Analyzer project, quality assurance practices were applied to minimize defects, improve reliability, and ensure accurate resume evaluation. Defect management activities were divided into three major categories:

1. Defect Prevention
2. Defect Reduction
3. Defect Containment

These activities helped improve the quality, usability, and maintainability of the system.

2. Defect Prevention Techniques

Defect prevention focuses on avoiding defects before they are introduced into the software.

2.1 Requirement Reviews

The Software Requirement Specification (SRS) document was reviewed by team members to ensure that all project requirements were clear, complete, and consistent.

Benefits

- Reduced requirement ambiguity
- Improved understanding among developers
- Minimized future rework

2.2 Agile Development Methodology

The project followed Agile practices where development was divided into smaller iterations.

Benefits

- Continuous feedback
- Early issue identification
- Improved adaptability to requirement changes

2.3 Design Reviews

System architecture, workflow diagrams, and module interactions were reviewed before implementation.

Benefits

- Identification of design flaws
- Better module integration planning
- Reduced architectural defects

2.4 Coding Standards

Developers followed Python coding standards and naming conventions.

Benefits

- Improved code readability
- Easier maintenance
- Reduced programming errors

2.5 Dataset Validation

Training data used for machine learning was manually verified and labeled.

Benefits

- Improved model accuracy
- Reduced classification errors
- Better prediction consistency

2.6 Input Validation

Validation rules were applied to uploaded files and user inputs.

Examples

- Accept only PDF files
- Restrict invalid characters
- Verify mandatory information

Benefits

- Reduced runtime errors
- Improved system security

3. Defect Reduction Techniques

Defect reduction focuses on identifying and removing defects that already exist in the software.

3.1 Unit Testing

Each module was tested independently.

Modules Tested

- Resume Upload Module
- PDF Parser
- Resume Scoring Engine
- Classification Engine
- Feedback Generator

Benefits

- Early defect detection
- Easier debugging

3.2 Integration Testing

Testing was conducted after combining individual modules.

Verified Interactions

- Frontend ↔ Backend
- Backend ↔ ML Model
- Parser ↔ Scoring Engine

Benefits

- Reduced interface defects
- Improved system communication

3.3 System Testing

The complete application was tested as a single system.

Test Scenarios

- Upload valid resume
- Upload invalid file
- Generate score
- Generate feedback
- Display classification result

Benefits

- Validation of end-to-end functionality
- Improved reliability

3.4 Code Reviews

Team members reviewed each other's code before integration.

Benefits

- Improved code quality
- Detection of logical errors
- Better maintainability

3.5 Machine Learning Model Validation

The Logistic Regression model was evaluated using test datasets.

Validation Activities

- Accuracy checking
- Prediction verification
- Output consistency analysis

Benefits

- Improved classification performance
- Reduced prediction errors

4. Defect Containment Techniques

Defect containment focuses on preventing existing defects from affecting end users.

4.1 Exception Handling

Try-except blocks were implemented in critical modules.

Areas Covered

- PDF parsing
- File uploading
- Data processing

Benefits

- Prevented application crashes
- Improved system stability

4.2 Error Messages

User-friendly error messages were displayed when issues occurred.

Examples

- Invalid PDF uploaded
- Unsupported file format
- Resume processing failure

Benefits

- Improved user experience
- Easier issue identification

4.3 File Validation

Uploaded files were validated before processing.

Validation Checks

- File type verification
- Empty file detection
- Corrupted file detection

Benefits

- Prevented invalid data processing
- Improved reliability

4.4 Logging and Monitoring

System activities and exceptions were recorded in logs.

Benefits

- Easier troubleshooting
- Faster defect resolution
- Improved maintenance

4.5 Controlled Deployment

The Flask application was deployed in a controlled environment before final release.

Benefits

- Reduced production failures
- Early issue detection

6. Corrective Actions

The following corrective actions were recommended:

1. Implement strict file validation.
2. Improve exception handling.
3. Enhance feedback personalization.
4. Add score breakdown visualization.
5. Expand machine learning training dataset.
6. Improve error reporting mechanisms.
7. Support additional resume formats.

7. Preventive Actions

To avoid future defects, the following actions are recommended:

1. Conduct formal inspections regularly.
2. Maintain coding standards.
3. Increase automated testing coverage.
4. Use Continuous Integration (CI) pipelines.
5. Perform regular code reviews.
6. Update machine learning datasets periodically.
7. Maintain comprehensive project documentation.

8. Quality Metrics

The following metrics can be used to evaluate software quality:

Defect Density

Number of defects per module.

Defect Removal Efficiency (DRE)

$DRE = \text{Defects Removed Before Release} / \text{Total Defects} \times 100$

Test Coverage

Percentage of requirements tested.

Defect Leakage

Defects discovered after deployment.

Mean Time to Resolve (MTTR)

Average time required to fix defects.

9. Conclusion

The AI Resume Analyzer project applied multiple quality assurance practices to improve software reliability and accuracy. Defect prevention activities reduced the introduction of errors, defect reduction techniques helped identify and remove existing issues, while defect containment mechanisms prevented failures from impacting users. These practices significantly improved the overall quality of the system and provided a strong foundation for future enhancements.